AULA 2

Estruturas de Controle e Laços de Repetição

Curso de Programação em Java

Módulo 1 - Fundamentos

18/04/2026

O que vamos aprender hoje?

Tomar decisões no código
(if/else, switch)

Repetir ações automaticamente
(for, while, do-while)

Criar programas muito mais
inteligentes e poderosos!

Estrutura Condicional – if

Sintaxe

```
if (condição) {
    // código executado se
    // condição for verdadeira
}
```

Condição é uma expressão que resulta em **true** ou **false**

Exemplo

```
int idade = 20;

if (idade ≥ 18) {
    System.out.println("Você é maior de idade!");
}
```

Resultado: "Você é maior de idade!"

if-else - Escolhendo Caminhos

```java
int idade = 16;

if (idade ≥ 18) {
    System.out.println("Você é maior de idade!");
} else {
    System.out.println("Você é menor de idade.");
}
```

Resultado

"Você é menor de idade."

Lembre-se:

if = se a condição é verdadeira
else = caso contrário

SEMPRE um dos dois blocos será executado

Operadores Relacionais

Operador	Significado	Exemplo
`==` Igual a	Ambos devem ser iguais	`5 == 5` → true
`≠` Diferente de	Devem ser diferentes	`5 ≠ 3` → true
`>` Maior que	Primeiro maior que segundo	`7 > 5` → true
`<` Menor que	Primeiro menor que segundo	`3 < 10` → true
`≥` Maior ou igual	Maior ou igual ao segundo	`5 ≥ 5` → true
`≤` Menor ou igual	Menor ou igual ao segundo	`4 ≤ 3` → false

⚠ Atenção!

`==` (comparação) é diferente de `=` (atribuição)

Erro Comum

```
if (x = 5)  // NÃO COMPILA!
```

Use `==` para comparar valores

Resultam sempre em **true** ou **false**

if-else if-else - Múltiplas Condições

Exemplo: Classificação de Notas

```java
double nota = 7.5;

if (nota ≥ 9.0) {
    System.out.println("Excelente!");
} else if (nota ≥ 7.0) {
    System.out.println("Bom!");
} else if (nota ≥ 5.0) {
    System.out.println("Regular.");
} else {
    System.out.println("Insuficiente.");
}
```

Resultado

"Bom!"

nota = 7.5

Como funciona:

1. Testa de cima para baixo
2. Para no **primeiro true**
3. **Ordem importa!**

9.0+ Excelente	7.0-8.9 Bom
5.0-6.9 Regular	0-4.9 Insuficiente

Operadores Lógicos

Operador	Nome	Significado
&& E (AND)	Ambos devem ser **true**	`a > 5 && b < 10`
|| OU (OR)	Pelo menos um **true**	`a > 5 || b < 10`
! NÃO (NOT)	Inverte o resultado	`!(a > 5)`

Exemplo Prático

```
int idade = 25;
boolean temCNH = true;

if (idade ≥ 18 && temCNH) {
  System.out.println("Pode dirigir!");
}
```

Para dirigir precisa:

Maior de 18 anos **E** ter CNH

&&
E também

||
Ou então

Laços de Repetição – for

Quando usar:

Quando sabemos quantas vezes repetir

Sintaxe:

```
for (inicialização; condição; incremento) {
    // código a ser repetido
}
```

Exemplo:

```
for (int i = 1; i ≤ 5; i++) {
    System.out.println("Contagem: " + i);
}
```

Resultado

1 2 3 4 5

Como funciona:

1. **int i = 1**
   Começa em 1
2. **i <= 5**
   Continua enquanto for verdadeiro
3. **i++**
   Incrementa a cada volta

for – Exemplo Tabuada

Programa completo:

```java
import java.util.Scanner;

public class Tabuada {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Digite um número: ");
    int numero = sc.nextInt();

    for (int i = 1; i ≤ 10; i++) {
      System.out.println(numero + " x " + i + " = " +
    }

    sc.close();
  }
}
```

Tabuada do 7

7 x 1 = 7	7 x 2 = 14
7 x 3 = 21	7 x 4 = 28
7 x 5 = 35	7 x 6 = 42
7 x 7 = 49	7 x 8 = 56
7 x 9 = 63	7 x 10 = 70

Vantagem do for:

Sem o **for**, teríamos que escrever **10 linhas repetidas**!

while – Repetição com Condição

Quando usar:

Quando **NÃO sabemos** quantas vezes repetir

Sintaxe:

```
while (condição) {
    // código a ser repetido
}
```

Exemplo:

```
int contador = 1;

while (contador ≤ 5) {
    System.out.println("Contador: " + contador);
    contador++;
}
```

Como funciona:

1. Verifica a **condição** primeiro
2. Se for **true**, executa o bloco
3. Volta ao passo 1 automaticamente
4. Quando for **false**, para o loop

Atenção!

Atualize a variável dentro do loop! Senão, loop infinito! ⚠

while – Exemplo Senha

Repetir até acertar

```java
String senhaCorreta = "java2026";
String senha = "";

while (!senha.equals(senhaCorreta)) {
    System.out.print("Digite a senha: ");
    senha = sc.nextLine();

    if (!senha.equals(senhaCorreta)) {
        System.out.println("Senha incorreta! Tente novamente.");
    }
}

System.out.println("Senha correta! Acesso liberado! ✅");
```

Como funciona:

- ! (NOT) inverte o resultado
- 1 Continua enquanto senha for **diferente**
- 2 Para quando acerta a senha

Vantagem do while:

Perfeito quando **não sabemos** quantas tentativas serão necessárias!

Loop Flexível

Repete até que a condição seja satisfeita

do-while - Executar Pelo Menos Uma Vez

Diferença:

Executa o bloco **PRIMEIRO**, depois verifica a condição

Sintaxe:

```
do {
    // código a ser repetido
} while (condição);
```

Garantia:

Sempre executa pelo menos **1 vez**, mesmo que a condição seja false

Exemplo:

```
int numero;

do {
    System.out.print("Digite um número positivo: ");
    numero = sc.nextInt();

    if (numero ≤ 0) {
        System.out.println("Erro! Deve ser positivo.");
    }
} while (numero ≤ 0);

System.out.println("Você digitou: " + numero);
```

Quando usar?

Quando precisa **executar pelo menos uma vez** antes de verificar

break e continue

break

Sai do laço imediatamente

Exemplo:

```
for (int i = 1; i ≤ 10; i++) {
    if (i == 5) {
        break; // para no 5
    }
    System.out.println(i);
}
```

Imprime: 1 2 3 4

continue

Pula para a próxima iteração

Exemplo:

```
for (int i = 1; i ≤ 5; i++) {
    if (i == 3) {
        continue; // pula o 3
    }
    System.out.println(i);
}
```

Imprime: 1 2 4 5

★ Pirâmide de Asteriscos

Exemplo prático com loops aninhados

```java
int altura = 5;

for (int i = 1; i ≤ altura; i++) {
    for (int j = 1; j ≤ i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

Resultado:

```
*
* *
* * *
* * * *
* * * * *
```

Como funciona?

Loop externo: **linhas** • Loop interno: **estrelas por linha**

Loop Externo
Controla as linhas

Loop Interno
Controla as estrelas

Recapitulando

Estruturas de Controle

`if, else, else if` → tomar decisões `switch` → múltiplas opções fixas

Laços de Repetição

`for` → quando sabemos quantas vezes repetir

`while` → enquanto condição for true

`do-while` → executa pelo menos uma vez

Controles

`break` → sai do laço `continue` → pula para próxima iteração

Próxima aula: Arrays e Strings (listas de dados!)